

# DATA 260

# HOMEWORK 1

**Shoury Ambarish Parab**

Repository: data260-7085

Topic: Local restaurant inspections

|                    |                                          |
|--------------------|------------------------------------------|
| <b>SID4</b>        | <b>7085</b>                              |
| <b>PORT_BASE</b>   | 8785                                     |
| <b>PREFIX</b>      | s7085                                    |
| <b>SEED</b>        | 7085                                     |
| <b>VERIFY_SEED</b> | 267085                                   |
| <b>DOMAIN_ID</b>   | 5                                        |
| <b>Python</b>      | 3.12                                     |
| <b>Local model</b> | llama3.2:3b                              |
| <b>Hardware</b>    | Intel(R) Core(TM) i5-6200U CPU @ 2.30GHz |

## 1. Part 1 — Web Application

The application was developed for my assigned local restaurant inspections domain. I used the domain schema to keep the form fields and categories tied to the assigned entity.

### 1.1 HTML / Domain Schema

Relevant HTML:

```
<input id="restaurantName" name="restaurantName" type="text" placeholder="Restaurant name" required
autofocus>

<input id="location" name="location" type="text" placeholder="Restaurant location" required>

<input id="email" name="email" type="email" placeholder="Submitter email" required>

<textarea id="description" name="description" placeholder="Describe the inspection findings"
required></textarea>

<select id="category" name="category" required>

  <option value="">Select inspection category</option>

  <option>Food safety</option>

  <option>Sanitation</option>

  <option>Inspection compliance</option>

  <option>Health code violation</option>

</select>

<label><input type="checkbox" id="terms" required> I agree to the terms and conditions.</label>

<button type="submit">Submit Inspection</button>

<script src="script.js"></script>
```

DOMAIN\_SCHEMA.md defines the domain entity and its fields/categories. index.html uses the defined domain information in the web form. HTML was used to build the basic structure of the application. I treated the page like a small data-entry form where a user can enter information about a restaurant inspection. I also used HTML's built-in form features wherever possible instead of depending entirely on JavaScript. This keeps the initial form behavior simple and gives the user feedback directly from the browser.

Files: DOMAIN\_SCHEMA.md, index.html

# Local Restaurant Inspections

## Restaurant Name

## Location

## Email

## Inspection Description

## Inspection Category

☐ I agree to the terms and conditions.

Submit Inspection

This screenshot shows the completed web form. The cursor is already in the restaurant name field demonstrating the autofocus behavior.

## 1.2 JavaScript Behavior

Relevant JavaScript:

```
const validateForm = (description, termsAccepted) => {  
  if (description.length <= 25) { alert('Description must be more than 25 characters.');
```

```
    return false; }  
  if (!termsAccepted) { alert('Please agree to the terms and conditions.');
```

```
    return false; }  
  return true;  
};  
  
const jsonData = JSON.stringify(formData);  
console.log(jsonData);  
  
const { restaurantName, email } = formData;
```

```

console.log(restaurantName, email);

const updatedData = { ...formData, submissionDate: new Date().toISOString() };

console.log(updatedData);

const submissionCounter = (() => {

  let count = 0;

  return () => ++count;

})();

```

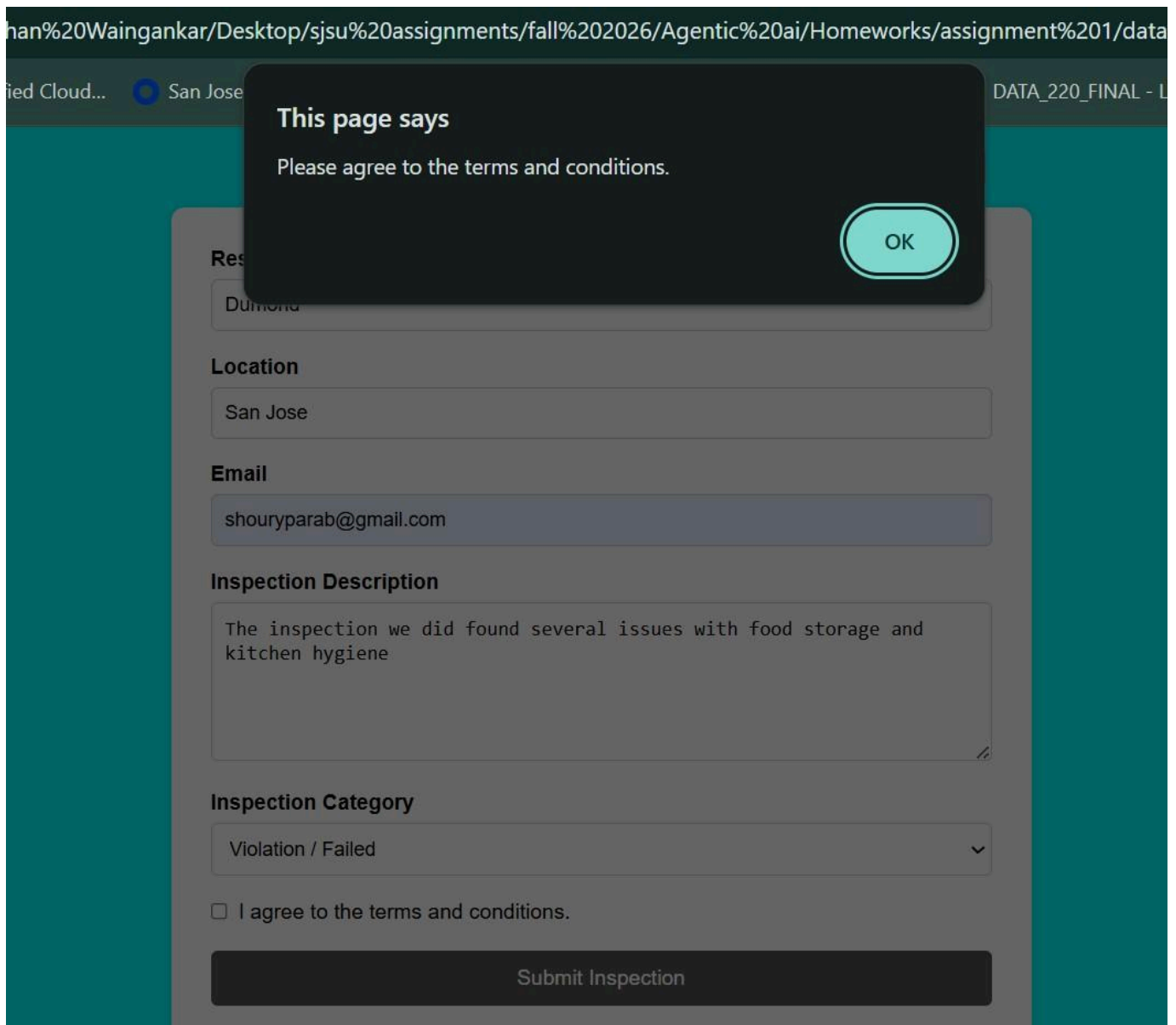
JavaScript was added to make the form interactive after the basic HTML structure was complete. The main idea was to take the information entered by the user, check it, and then work with it as JavaScript data. This also gave me a chance to use some common JavaScript features in an actual application instead of using them only as isolated examples.

File: script.js

The screenshot shows a web browser window with a teal header. The address bar displays a file path: `Bhushan%20Waingankar/Desktop/sjsu%20assignments/fall%202026/Agentic%20ai/Homeworks/assignment%201/data2`. The browser tabs include "Certified Cloud...", "San Jose", and "DATA\_220\_FINAL - L...". A dark modal box is centered on the screen with the text "This page says" and "Inspection description must be more than 25 characters." with an "OK" button. The background form is titled "Res..." and contains the following fields:

- Restaurant Name:** A text input field containing "Dumond".
- Location:** A text input field containing "San Jose".
- Email:** A text input field containing "shouryparab@gmail.com".
- Inspection Description:** A text area containing "Decent food".
- Inspection Category:** A dropdown menu with "Conditional Pass" selected.
- Terms and Conditions:** A checkbox labeled "I agree to the terms and conditions." which is checked.
- Submit Button:** A dark button labeled "Submit Inspection".

The form rejects the submission because the review text contains fewer than 25 characters demonstrating the required client-side length check.



The screenshot shows a web browser window with a teal header. A modal dialog box is centered on the screen, titled "This page says" with the message "Please agree to the terms and conditions." and an "OK" button. The background is a registration form with the following fields:

- Residence:** A text input field containing "Dumond".
- Location:** A text input field containing "San Jose".
- Email:** A text input field containing "shouryparab@gmail.com".
- Inspection Description:** A text area containing "The inspection we did found several issues with food storage and kitchen hygiene".
- Inspection Category:** A dropdown menu showing "Violation / Failed".
- Terms and Conditions:** A checkbox labeled "I agree to the terms and conditions." which is currently unchecked.
- Submit Button:** A dark button labeled "Submit Inspection".

The form is rejected because the terms andconditions checkbox is not selected.

### This page says

Inspection submitted successfully!

OK

#### Res

Diamond

#### Location

San Jose

#### Email

shouryparab@gmail.com

#### Inspection Description

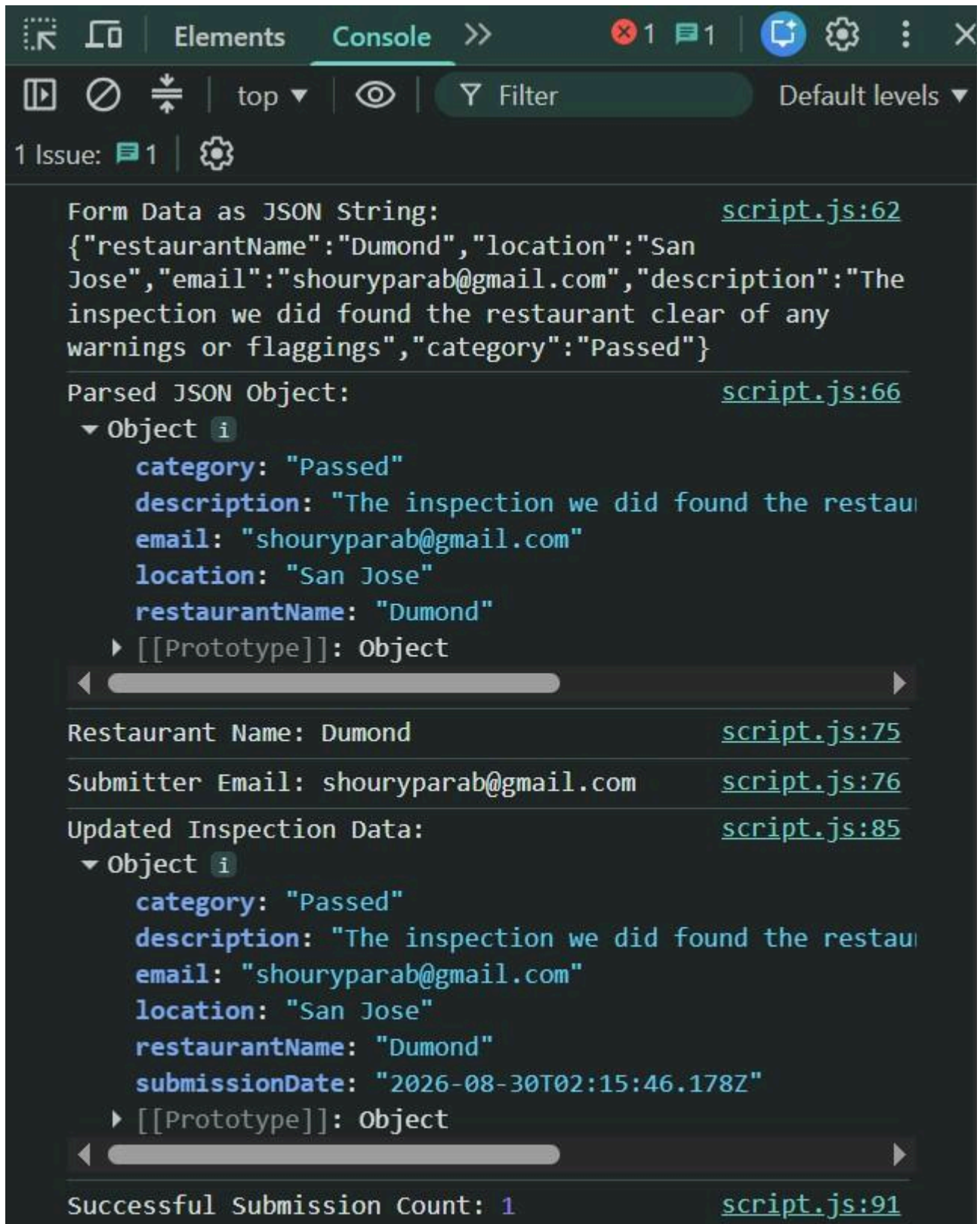
The inspection we did found the restaurant clear of any warnings or  
flaggings

#### Inspection Category

Passed

☒ I agree to the terms and conditions.

Submit Inspection



The screenshot shows a web browser's developer console with the 'Console' tab selected. At the top, there are icons for 'Elements', 'Console', and a search icon. Below these, there are icons for 'top', 'Filter', and 'Default levels'. The console shows 1 issue. The log contains the following messages:

```
Form Data as JSON String: script.js:62
{"restaurantName":"Dumond","location":"San Jose","email":"shouryparab@gmail.com","description":"The inspection we did found the restaurant clear of any warnings or flaggings","category":"Passed"}

Parsed JSON Object: script.js:66
  ▼ Object i
    category: "Passed"
    description: "The inspection we did found the restaura
    email: "shouryparab@gmail.com"
    location: "San Jose"
    restaurantName: "Dumond"
    ▶ [[Prototype]]: Object

Restaurant Name: Dumond script.js:75
Submitter Email: shouryparab@gmail.com script.js:76
Updated Inspection Data: script.js:85
  ▼ Object i
    category: "Passed"
    description: "The inspection we did found the restaura
    email: "shouryparab@gmail.com"
    location: "San Jose"
    restaurantName: "Dumond"
    submissionDate: "2026-08-30T02:15:46.178Z"
    ▶ [[Prototype]]: Object

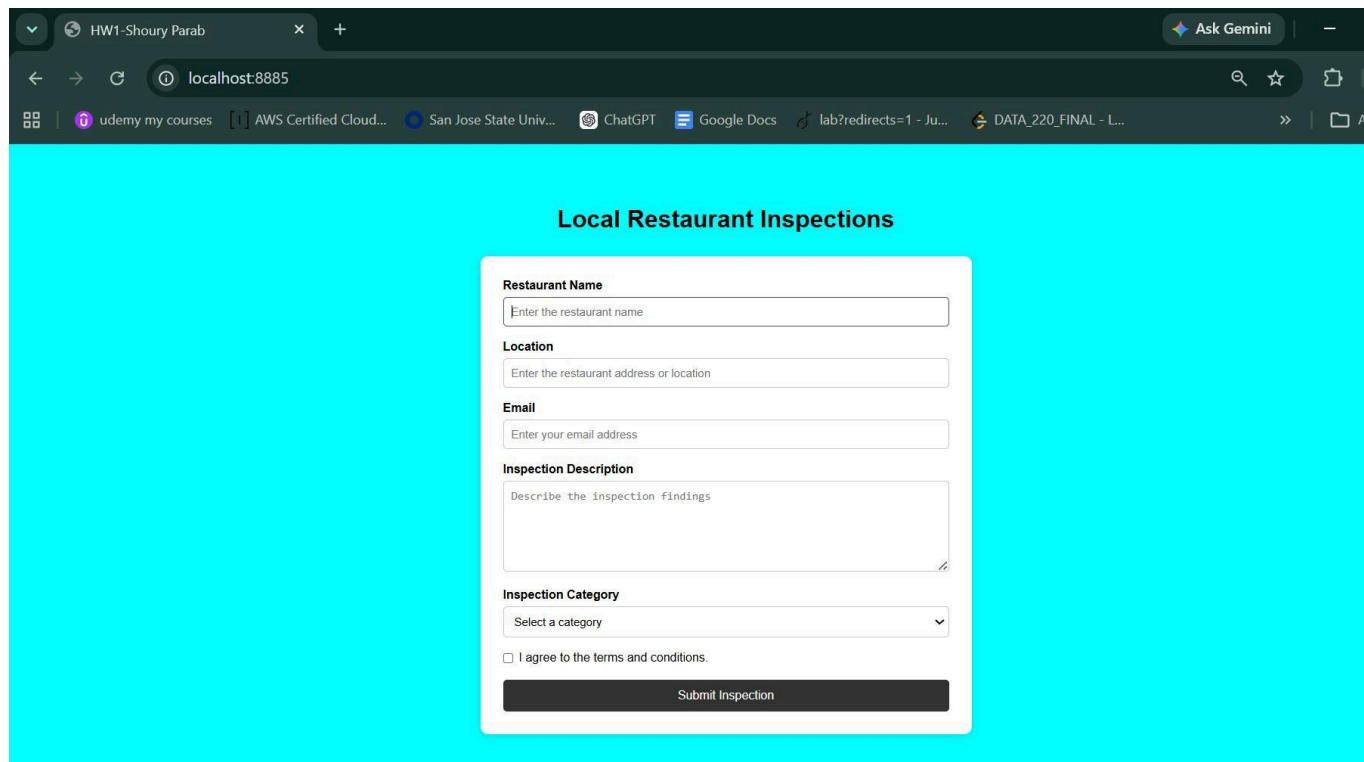
Successful Submission Count: 1 script.js:91
```

The valid submission is processed successfully. The console shows the processed data and the submission counter increasing.

## 1.3 Docker / Local Deployment

After getting the application working locally, I used Docker to package it into a container. I found this useful because the application and its runtime setup can be treated as one unit. The local Docker test was also a good way to make sure the application still worked outside the normal development setup.

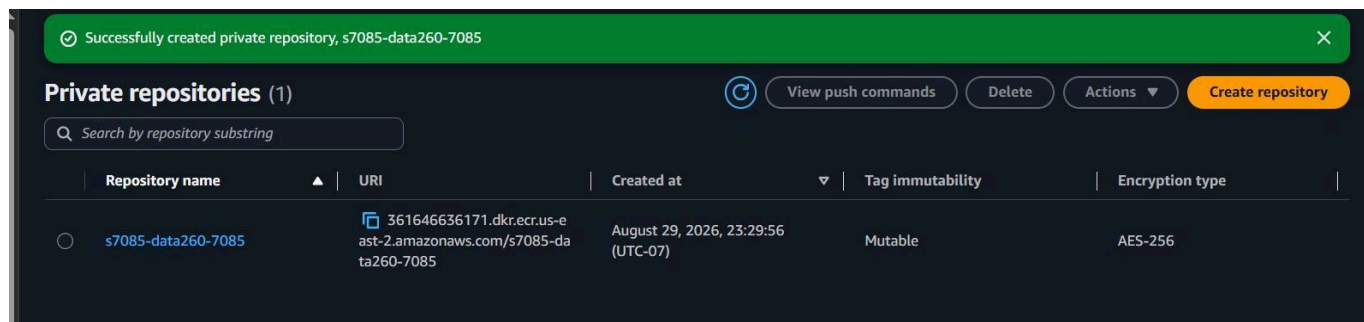
Dockerfile



**What is happening:** The containerized application is running on localhost.

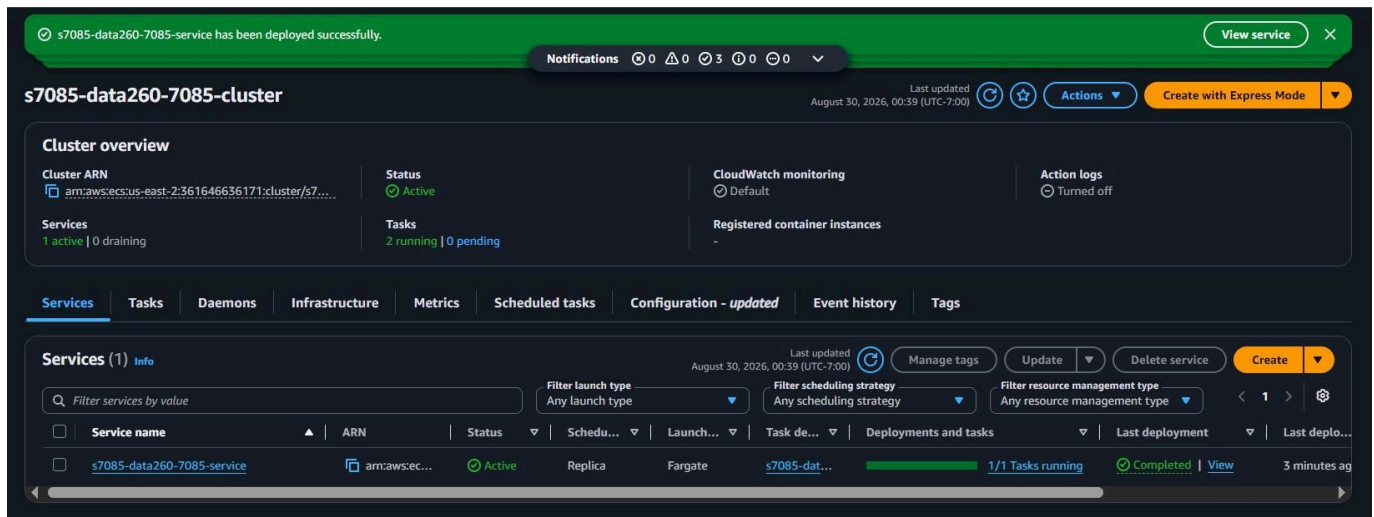
## 1.4 AWS Deployment

The next step was moving the same containerized application to AWS. ECS provides a way to run the Docker container as a service instead of keeping it only on my computer. The public IP test was useful because it showed that the application could actually be reached from outside my local environment.

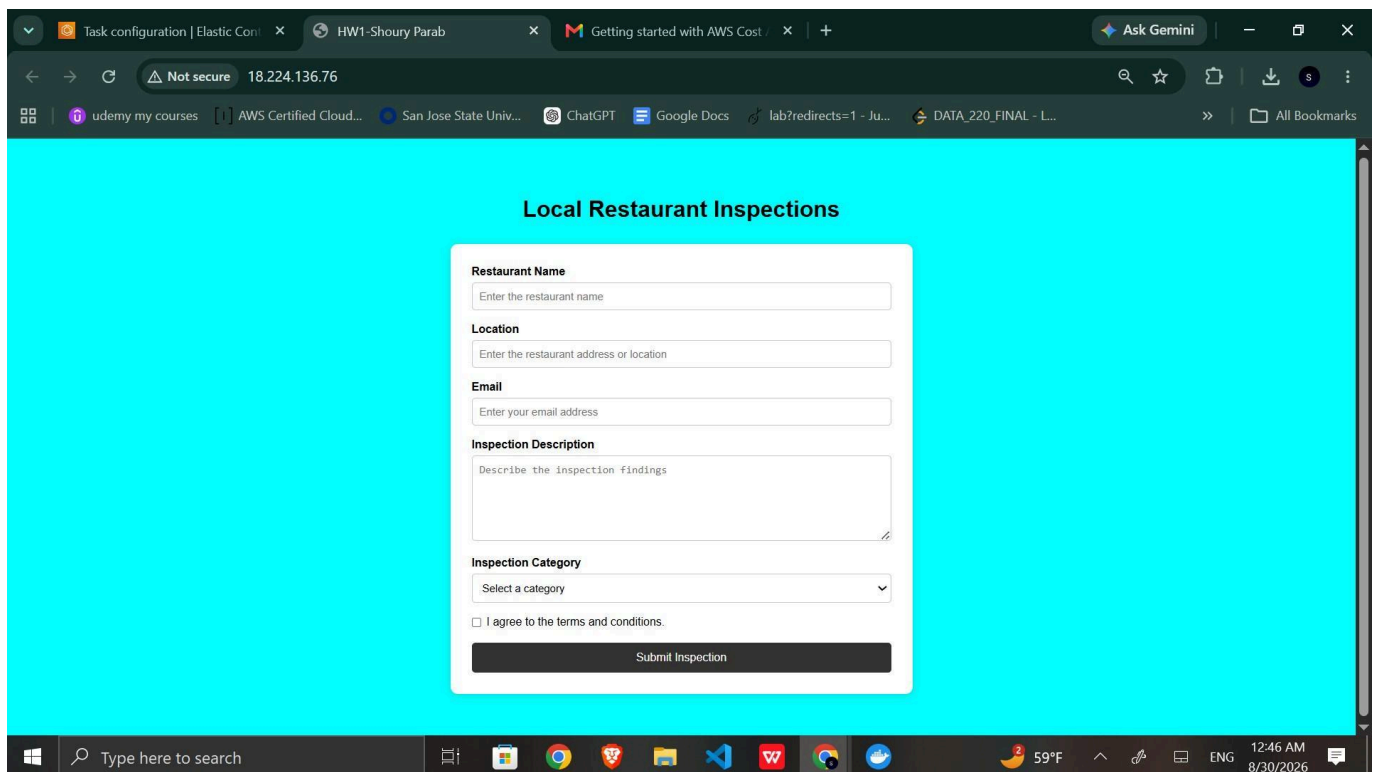


The Docker image repository is visible in Amazon ECR, showing that the image was pushed to the AWS container registry.





The ECS service deployment is shown as completed, providing evidence that the container was successfully deployed as the required service.



The application is accessible through the public AWS deployment, confirming that the deployed service is reachable outside the local environment.

## 2. Part 2 — Agentic AI

The agent section was different from the web application because the goal was to have multiple stages of model reasoning instead of relying on one response. I used the Planner to create an initial result and the

Reviewer to check that result before the final output was produced. This makes the workflow easier to understand because each stage has a specific job.

## 2.1 Exact Command

The command used for the agent demonstration:

```
python agent_demo.py --title "Restaurant Inspection Report" --content "A local restaurant inspection evaluates food safety, sanitation, hygiene, and compliance with public health requirements."
```

## 2.2 Planner Output

**Theory:** I think of this workflow as a small review pipeline. The Planner makes the first attempt, the Reviewer looks for problem and the Finalizer turns the reviewed information into the final format. Separating the steps also makes it easier to see where an incorrect or incomplete result could be caught before it reaches the final output.

```
ation for submitting and reviewing local restaurant inspection findings, including restaurant name, location, email, inspection description, and insp

===== PLANNER OUTPUT =====
{
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews"
}

===== REVIEWER OUTPUT =====
{
  "approved": true,
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews",
  "issues": []
}

===== FINALIZED OUTPUT =====
{
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews"
}
PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> |
```

The console shows the agent pipeline running and the Planner stage producing its initial proposal.

## 2.3 Reviewer Output

The Reviewer examines the Planner's proposal before the Finalizer produces the final result. This provides a separate checking step rather than accepting the first model output automatically.

```

PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> .\venv\Scripts\python.exe agent_demo.py --title "Local Restaurant Inspections" --content "A web application for submitting and reviewing local restaurant inspection findings, including restaurant name, location, email, inspection description, and inspection category."

===== PLANNER OUTPUT =====
{
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews"
}

===== REVIEWER OUTPUT =====
{
  "approved": true,
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews",
  "issues": []
}

===== FINALIZED OUTPUT =====
{
  "tags": [
    "restaurant inspections",
    "local government",
    "food safety"
  ],
  "summary": "Web app for local restaurant inspection submissions and reviews"
}
PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>

```

The console shows the Planner and Reviewer stages of the agent conversation, including the intermediate proposal and its review.

## 2.4 Finalized Output

Theory: The finalizer is the last stage, so its output is the one that should be checked against the exact formatting constraints before being treated as the final answer.

The Finalizer takes the reviewed result and produces the final JSON that can be treated as the completed output.

Q1 — Final tags: Restaurant Inspection, Local Business, Web Application

Q2 — Final summary : The web application facilitates the submission and checking of local restaurant inspections.

Q3 — Did the Reviewer change anything? : No. The Reviewer approved the Planner's result and echoed the same tags/summary; the Finalizer then produced the final output. The recorded implementation explicitly has the Reviewer validate and the Finalizer use the reviewer feedback.

Step explanation: The Planner creates the candidate tags and summary, the Reviewer checks that proposal, and the Finalizer produces the final JSON after the review step.

## 3. Part 3 — Measuring Non-Determinism

I used one fixed domain relevant input stored in reports/hw01/cases/nondeterminism\_input.json for all 40 runs. I kept the input unchanged and only varied the temperature: 20 runs at 0.7 and 20 runs at 0.0.

### 3.1 Experiment Evidence

Experiment structure:

for temperature in [0.7, 0.0]:

run the same fixed input 20 times

record latency\_ms and final\_output for every run

save the results for later analysis

Theory: Repeating the same input makes it possible to separate model-output variation from changes in the input. Comparing the two temperature settings gives a controlled way to study how generation settings affect consistency and response time.

```
rant inspections" --content "A web application for submitting and reviewing local restaurant inspection findings, including restaurant name, location, email, inspection description
d inspection category." --temperature 0.7

===== PLANNER OUTPUT =====
{
  "tags": [
    "restaurant inspection",
    "local restaurant",
    "rating system"
  ],
  "summary": "Application for submitting and reviewing local restaurant inspection findings online"
}

===== REVIEWER OUTPUT =====
{
  "approved": true,
  "tags": [
    "restaurant inspection",
    "local restaurant inspection",
    "rating system"
  ],
  "summary": "Web app for submitting and reviewing local restaurant inspection findings",
  "issues": []
}

===== FINALIZED OUTPUT =====
{
  "tags": [
    "restaurant inspection",
    "local restaurant",
    "rating system"
  ],
  "summary": "Web app for submitting and reviewing local restaurant inspection findings online"
}
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>
```

The temperature 0.7 experiment is running with the fixed input.

Starting experiment...

--- Temperature 0.7 ---

Run 01/20 (temperature=0.7)...

Run 02/20 (temperature=0.7)...

Run 03/20 (temperature=0.7)...

Run 04/20 (temperature=0.7)...

Run 05/20 (temperature=0.7)...

Run 06/20 (temperature=0.7)...

Run 07/20 (temperature=0.7)...

Run 08/20 (temperature=0.7)...

Run 09/20 (temperature=0.7)...

Run 10/20 (temperature=0.7)...

Run 11/20 (temperature=0.7)...

Run 12/20 (temperature=0.7)...

Run 13/20 (temperature=0.7)...

Run 14/20 (temperature=0.7)...

Run 15/20 (temperature=0.7)...

Run 16/20 (temperature=0.7)...

Run 17/20 (temperature=0.7)...

Run 18/20 (temperature=0.7)...

Run 19/20 (temperature=0.7)...

Run 20/20 (temperature=0.7)...

--- Temperature 0.0 ---

Run 01/20 (temperature=0.0)...

Run 02/20 (temperature=0.0)...

Run 03/20 (temperature=0.0)...

Run 04/20 (temperature=0.0)...

Run 05/20 (temperature=0.0)...

Run 06/20 (temperature=0.0)...

Run 07/20 (temperature=0.0)...

Run 08/20 (temperature=0.0)...

Run 09/20 (temperature=0.0)...

Run 10/20 (temperature=0.0)...

Run 11/20 (temperature=0.0)...

Run 12/20 (temperature=0.0)...

Run 13/20 (temperature=0.0)...

Run 14/20 (temperature=0.0)...

Run 15/20 (temperature=0.0)...

Run 16/20 (temperature=0.0)...

Run 17/20 (temperature=0.0)...

Run 18/20 (temperature=0.0)...

Run 19/20 (temperature=0.0)...

Run 20/20 (temperature=0.0)...

=====

EXPERIMENT COMPLETE

=====

Total runs: 40

Raw JSON: reports\hw01\raw\nondeterminism\_results.json

Raw CSV: reports\hw01\raw\nondeterminism\_results.csv

The full set of 40 experiment runs has been completed.







```

=== TAG FREQUENCY ANALYSIS ===

TEMPERATURE: 0.7
Runs: 20
Distinct tag sets: 18
Tag frequencies:
  restaurant inspections : 13
  local government : 8
  local restaurant : 8
  restaurant inspection : 7
  web application : 7
  restaurant management : 4
  food safety : 2
  local restaurant inspections : 2
  local business : 1
  restaurant review : 1
  web applications : 1
  inspection report : 1
  inspections : 1
  reviews : 1
  inspections and reviews : 1
  restaurant inspection data : 1
  local restaurants : 1
Tags appearing in all 20 runs:
Tags appearing exactly once:
  local business
  restaurant review
  web applications
  inspection report
  inspections
  reviews
  inspections and reviews
  restaurant inspection data
  local restaurants
Tag sets appearing exactly once: 16

TEMPERATURE: 0.0
Runs: 20
Distinct tag sets: 1
Tag frequencies:
  restaurant inspections : 20
  local government : 20
  food safety : 20
Tags appearing in all 20 runs:
  restaurant inspections
  local government
  food safety
Tags appearing exactly once:
Tag sets appearing exactly once: 0

=== DONE ===

```

The tag-frequency analysis compares the tag sets produced at both temperatures.



```

77 print('Temperature 0.7 vs 0.0 latency difference: {difference:2f}%')
>> '@ | .\venv\Scripts\python.exe -ar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>
Temperature 0.7
Mean latency: 52313.27 ms

Temperature 0.0
Mean latency: 43349.38 ms

Temperature 0.7 vs 0.0 latency difference: 20.68%
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>

```

The mean latency values for the two temperature settings are compared.

## 3.2 Results

| Metric              | Temperature 0.7 | Temperature 0.0                                       |
|---------------------|-----------------|-------------------------------------------------------|
| Runs                | 20              | 20                                                    |
| Distinct tag sets   | 18              | 1                                                     |
| Mean latency (ms)   | 52313.27        | 43349.38                                              |
| p50 (ms)            | 50517.81        | 43025.84                                              |
| p95 (ms)            | 55879.40        | 43942.77                                              |
| p99 (ms)            | 81676.63        | 47393.76                                              |
| Tags in all 20 runs | None            | restaurant inspections; local government; food safety |
| Tags exactly once   | 9 unique tags   | None                                                  |

Interpretation: At temperature 0.7, the experiment produced 18 distinct tag sets across 20 runs, while temperature 0.0 produced only one tag set. This is a clear difference in repeatability for the same input. Mean latency was also higher at 0.7 by 20.68%. Two users sending identical input could therefore receive different tags or wording at the higher temperature. That variation can be useful for brainstorming but it would be less desirable for compliance or safety-related outputs where repeatability matters.

The key observation is that the same input was much less repeatable at temperature 0.7 than at 0.0, while the measured latency was also higher in this run.

The 18 different tag sets at 0.7 show that repeated identical inputs did not consistently produce the same tag set. The single tag set at 0.0 was much more repeatable.

## 4. Part 4 — Model Client and Token Accounting

### 4.1 Reusable Model Adapter

Relevant adapter interface:

```

def complete(self, messages, tools=None):

    # send the supplied conversation to the configured local model

    # return the model response and usage information

```

The model adapter keeps the model-call logic in one reusable place. `hw1_client.py` imports that adapter instead of duplicating the underlying model communication, which makes the client easier to reuse and maintain.

```
python hw1_client.py --message "Reply with one short sentence explaining what a restaurant inspection is."
```

```

(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> .\venv\Scripts\python.exe -m py_compile hw1_client.py
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> .\venv\Scripts\python.exe hw1_client.py --message "Reply with
one short sentence explaining what a restaurant inspection is."
{
  "model": "llama3.2:3b",
  "response": "A restaurant inspection is a thorough evaluation of a restaurant's food safety, sanitation, and overall compliance with health and safety regulations."
}
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>

```

A one-shot request is sent through the reusable model client and the local llama3.2:3b model returns a response. This confirms that the CLI is using the shared adapter successfully.

## 4.2 AGENT.md Code Review

Relevant client behavior:

load AGENT.md instructions

load review\_sample.py

send both as model context

print the returned code review

AGENT.md defines the required bullet-only code-review format. The first test did not produce a suitable review, so I corrected the client invocation so the instructions and review file were supplied explicitly. I then reran the test and checked the result against the instructions.

```

(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> .\venv\Scripts\python.exe -m py_compile hw1_client.py
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085> .\venv\Scripts\python.exe hw1_client.py --message "Review
s code for correctness and maintainability." --file review_sample.py
• The 'add_numbers' function does not handle cases where 'a' or 'b' are not numbers, which could lead to a TypeError.
• This matters because it could cause the program to crash unexpectedly, leading to a poor user experience.
• Improvement: Add input validation to ensure 'a' and 'b' are numbers before performing the addition.

• The 'greet' function does not handle cases where 'name' is empty or None, which could lead to an IndexError or a TypeError.
• This matters because it could cause the program to crash unexpectedly, leading to a poor user experience.
• Improvement: Add input validation to ensure 'name' is a non-empty string before printing the greeting.

• The 'add_numbers' function does not provide any documentation or comments, making it difficult for others to understand its purpose and behavior.
• This matters because it makes the code harder to maintain and understand for others.
• Improvement: Add docstrings to the 'add_numbers' function to explain its purpose and behavior.

• The 'greet' function does not provide any documentation or comments, making it difficult for others to understand its purpose and behavior.
• This matters because it makes the code harder to maintain and understand for others.
• Improvement: Add docstrings to the 'greet' function to explain its purpose and behavior.
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>

```

The corrected client produces the requested review of review\_sample.py using the instructions in AGENT.md. This provides evidence that the instruction file and the supplied code file were both being used.

The first unsuitable result is also documented in AI\_USE.md as the example of an AI output that needed correction and verification.

## 4.3 Token Accounting

Relevant accounting logic:

```
input_tokens = usage.get('prompt_eval_count', 0)
```

```
output_tokens = usage.get('eval_count', 0)
```

```
total_tokens = input_tokens + output_tokens
```

Theory: Token counts make the cost of a model interaction visible. Looking at both individual and cumulative counts is especially useful in a conversation because earlier messages can become part of later requests.

```
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\homeworks\assignment 1\data260-7085> .\.venv\Scripts\python.exe hml_client.py --message "one short sentence explaining what a restaurant inspection is."
{
  "model": "llama3.2:3b",
  "response": "A restaurant inspection is a thorough evaluation of a restaurant's food safety, sanitation, and overall compliance with health and safety regulations."
}

--- TOKEN USAGE ---
Input tokens: 37
Output tokens: 26
Total tokens: 63
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignments\fall 2026\Agentic ai\Homeworks\assignment 1\data260-7085>
```

The model client reports input, output, and total token counts after the response, making the token usage visible for each interaction.

## 4.4 Five-Turn Conversation

Relevant conversation structure:

```
history.append(user_message)

response = client.complete(history)

history.append(assistant_message)

history is reused for the next turn

if (command === '/stats') {

    console.log(turnCount, cumulativeInputTokens, cumulativeOutputTokens);

    console.log(JSON.stringify(history).length);

}
```

```
----- HML CLIENT -----
Model: llama3.2:3b
Type /stats to view statistics.
Type /exit to quit.
-----

You: What is a restaurant inspection?

Assistant: A restaurant inspection is a regulatory process conducted by local health departments or other government agencies to evaluate the cleanliness, safety, and overall quality of a restaurant. The purpose of a restaurant inspection is to ensure that the establishment is complying with health and safety regulations, and to identify any potential hazards or violations.

During a restaurant inspection, a health inspector will typically:

1. Observe the restaurant's food handling and preparation practices, including food storage, handling, and disposal.
2. Check the cleanliness and sanitation of the restaurant, including the kitchen, dining area, restrooms, and outdoor areas.
3. Verify that the restaurant is following proper food safety procedures, such as proper temperature control, cross-contamination prevention, and proper waste disposal.
4. Check the restaurant's equipment, including cooking equipment, dishwashers, and refrigerators, to ensure they are in good working condition and properly maintained.
5. Review the restaurant's food safety training records and ensure that all employees are properly trained on food safety procedures.

The inspector will typically use a checklist or scoring system to identify any violations or non-compliances. If the restaurant receives a passing grade, it is typically issued a certificate or permit that indicates it is in compliance with health and safety regulations.

Common reasons for a restaurant inspection include:

* Complaints from customers or employees
* Reports of foodborne illness or other health concerns
* Unusual odors or pests
* Failure to follow proper food safety procedures
* Non-compliance with health and safety regulations

If a restaurant receives a failing grade, it may be required to:

* Correct the identified violations
* Re-inspect to ensure compliance
* Pay a fine
* Close the restaurant temporarily or permanently

Overall, the goal of a restaurant inspection is to ensure that the establishment is providing a safe and healthy dining experience for its customers.

--- TOKEN USAGE ---
Input tokens: 31
Output tokens: 374
Total tokens: 405
```

This screenshot shows the first turn of the interactive conversation.

```
Problems Output Debug Console Terminal Ports
You: What are the main things inspectors check?

Assistant: During a restaurant inspection, health inspectors typically check for compliance with health and safety regulations, focusing on the following areas:

1. Food Handling and Preparation:
    * Proper food storage and handling procedures
    * Temperature control of hot and cold foods
    * Cross-contamination prevention
    * Proper food safety training for employees
2. Cleanliness and Sanitation:
    * Cleanliness of the kitchen, dining area, restrooms, and outdoor areas
    * Proper disposal of trash and recyclables
    * Cleaning and sanitizing of equipment and surfaces
3. Food Safety Procedures:
    * Proper handling and storage of raw meat, poultry, and seafood
    * Use of utensils and equipment to prevent cross-contamination
    * Proper cooking and reheating procedures
4. Equipment and Facilities:
    * Proper maintenance and cleaning of equipment, including dishwashers, refrigerators, and cooking equipment
    * Adequate ventilation and lighting
    * Proper drainage and sewage systems
5. Employee Hygiene and Training:
    * Proper handwashing and personal hygiene practices
    * Food safety training for employees
    * Employee attire and grooming
6. Food Allergens and Special Diets:
    * Proper labeling and handling of food allergens
    * Availability of special diets, such as gluten-free or vegan options
7. Pest Control and Rodent Prevention:
    * Presence of pests, such as cockroaches, rodents, or flies
    * Proper pest control measures
8. Waste Disposal and Recycling:
    * Proper disposal of food waste and recyclables
    * Adequate storage of hazardous waste
9. Temperature Control:
    * Proper temperature control of hot and cold foods
    * Use of thermometers to monitor temperatures
10. Compliance with Regulations:
    * Compliance with local health and safety regulations
    * Presence of required permits and licenses

Inspectors may also check for other specific requirements, such as:

* Proper food storage and handling of infant formula and other special diets
* Use of gloves and other personal protective equipment
* Proper cleaning and sanitizing of utensils and equipment
* Presence of a first aid kit and emergency procedures

The specific items checked may vary depending on the jurisdiction and the type of establishment being inspected.

--- TOKEN USAGE ---
Input tokens: 422
Output tokens: 483
Total tokens: 905
```

This screenshot shows the second turn of the interactive conversation.

You: Why are food-safety violations important?

Assistant: Food-safety violations are important because they can pose a risk to public health. When food is not handled, prepared, and served safely, it can contain pathogens, such as bacteria, viruses, and parasites, that can cause foodborne illnesses.

Foodborne illnesses can be serious and even life-threatening, especially for vulnerable populations such as:

1. **Infants and young children**: Their immune systems are still developing, and they may not be able to fight off infections as effectively.
2. **Older adults**: Their immune systems may be weakened, making them more susceptible to illness.
3. **Pregnant women**: Foodborne illnesses can pose a risk to the developing fetus.
4. **People with weakened immune systems**: Those with conditions such as HIV/AIDS, cancer, or taking immunosuppressive medications may be more susceptible to foodborne illnesses.

Food-safety violations can lead to a range of illnesses, including:

1. **Gastroenteritis**: Diarrhea, vomiting, and stomach cramps.
2. **Salmonellosis**: Fever, diarrhea, and abdominal cramps.
3. **E. coli**: Severe diarrhea, vomiting, and abdominal cramps.
4. **Listeriosis**: Fever, headache, and stiffness, which can be life-threatening in pregnant women and people with weakened immune systems.

The consequences of food-safety violations can be severe, including:

1. **Food poisoning outbreaks**: Large numbers of people can become ill, leading to hospitalizations and even deaths.
2. **Economic losses**: Foodborne illnesses can result in lost productivity, medical expenses, and other economic costs.
3. **Reputation damage**: Food-safety violations can damage a restaurant's or food establishment's reputation, leading to lost business and revenue.

To prevent food-safety violations, it's essential to:

1. **Follow proper food handling and preparation procedures**.
2. **Maintain a clean and sanitary environment**.
3. **Use safe food storage and handling practices**.
4. **Train employees on food safety procedures**.
5. **Regularly inspect and maintain equipment**.

By prioritizing food safety, we can reduce the risk of foodborne illnesses and protect public health.

--- TOKEN USAGE ---

Input tokens: 922

Output tokens: 450

Total tokens: 1372

You: /stats

This screenshot shows the third turn of the interactive conversation.

--- TOKEN USAGE ---

Input tokens: 922

Output tokens: 450

Total tokens: 1372

You: /stats

===== /stats =====

Turn count: 3

Cumulative input tokens: 1375

Cumulative output tokens: 1307

Serialized conversation-history length: 6940 characters

=====

The /stats command after turn 3 reports the current turn count, cumulative token usage and serialized conversation history length.



You: How can a restaurant prepare for an inspection?

Assistant: To prepare for a restaurant inspection, it's essential to take proactive steps to ensure that your establishment is clean, safe, and compliant with health and safety regulations. Here are some steps you can take:

1. **Review and update your food safety plan**: Ensure that your food safety plan is up-to-date and includes procedures for handling, storage, and disposal of food, as well as employee training and sanitation protocols.
2. **Conduct a self-inspection**: Schedule a self-inspection to identify potential areas of non-compliance and make necessary corrections before the inspector arrives.
3. **Clean and sanitize**: Thoroughly clean and sanitize all areas of the restaurant, including the kitchen, dining area, restrooms, and outdoor spaces.
4. **Organize and declutter**: Ensure that all areas of the restaurant are organized and clutter-free, including storage areas, walk-in refrigerators, and dishrooms.
5. **Check equipment and appliances**: Ensure that all equipment and appliances are in good working condition and properly maintained, including dishwashers, refrigerators, and cooking equipment.
6. **Verify employee training**: Ensure that all employees have received proper food safety training and are aware of their responsibilities and procedures.
7. **Review and update menus and recipes**: Ensure that all menus and recipes are up-to-date and compliant with food safety regulations.
8. **Check food storage and handling**: Ensure that all food is stored and handled properly, including proper labeling, dating, and rotation of inventory.
9. **Verify pest control measures**: Ensure that all pest control measures are in place and effective, including traps, baits, and cleaning schedules.
10. **Gather necessary documents**: Ensure that all necessary documents, including food safety certifications, permits, and licenses, are up-to-date and readily available.

**Additional Tips**

1. **Schedule a pre-inspection visit**: Consider scheduling a pre-inspection visit with the inspector to discuss any concerns or questions you may have.
2. **Be prepared to answer questions**: Be prepared to answer questions about your food safety procedures, employee training, and any concerns or issues you may have.
3. **Have a plan for corrective actions**: Have a plan in place for corrective actions in case of any non-compliances or issues identified during the inspection.
4. **Stay organized and focused**: Stay organized and focused during the inspection, and be prepared to address any concerns or issues that may arise.

**Timeline**

1. **1-2 weeks before the inspection**: Conduct a self-inspection and make any necessary corrections.
2. **1-3 days before the inspection**: Clean and sanitize all areas of the restaurant.
3. **Day of the inspection**: Review and update your food safety plan, and ensure that all employees are aware of their responsibilities and procedures.
4. **During the inspection**: Be prepared to answer questions and address any concerns or issues that may arise.

By following these steps and tips, you can help ensure that your restaurant is prepared for the inspection and can demonstrate compliance with health and safety regulations.

--- TOKEN USAGE ---  
Input tokens: 1390  
Output tokens: 617  
Total tokens: 2007

This screenshot shows the fourth turn of the interactive conversation.

You: What happens if a restaurant fails an inspection?

Assistant: If a restaurant fails an inspection, it can result in a range of consequences, including:

1. **Fines and penalties**: The restaurant may be required to pay fines and penalties for non-compliance with health and safety regulations.
2. **Closure**: In severe cases, the restaurant may be required to close temporarily or permanently until all non-compliances are corrected.
3. **Warning letters**: The restaurant may receive warning letters from the health department, outlining the specific issues that need to be corrected.
4. **Corrective action plans**: The restaurant may be required to develop a corrective action plan to address the specific issues that led to the failure of the inspection.
5. **Re-inspection**: The restaurant may be required to undergo a re-inspection to ensure that all non-compliances have been corrected.
6. **Loss of licenses or permits**: In extreme cases, the restaurant may lose its licenses or permits, making it impossible to operate.
7. **Negative publicity**: A failed inspection can result in negative publicity, including negative reviews and word-of-mouth, which can harm the restaurant's reputation and business.
8. **Employee disciplinary action**: The restaurant may be required to take disciplinary action against employees who are found to be responsible for the non-compliances.
9. **Increased costs**: The restaurant may need to incur additional costs to correct the non-compliances, including hiring additional staff or purchasing new equipment.

**Common Consequences of a Failed Inspection**

1. **First-time offenders**: May receive a warning letter and be required to correct the non-compliances within a specified timeframe.
2. **Repeat offenders**: May face more severe consequences, including fines, penalties, and closure.
3. **Critical violations**: May result in immediate closure and fines.
4. **Recurring issues**: May require the restaurant to develop a corrective action plan and undergo regular re-inspections.

**How to Appeal a Failed Inspection**

1. **Request a hearing**: The restaurant may request a hearing to appeal the failed inspection.
2. **Provide evidence**: The restaurant may provide evidence to support its appeal, such as documentation of corrective actions taken.
3. **Negotiate a settlement**: The restaurant may negotiate a settlement with the health department, such as a reduced fine or penalty.

It's essential for restaurants to take food safety seriously and to be proactive in preventing non-compliances. If a restaurant fails an inspection, it's crucial to take immediate action to correct the issues and prevent future failures.

--- TOKEN USAGE ---  
Input tokens: 2025  
Output tokens: 505  
Total tokens: 2530

This screenshot shows the fifth turn of the interactive conversation.



```
--- TOKEN USAGE ---
Input tokens: 2025
Output tokens: 505
Total tokens: 2530

You: /stats

===== /stats =====
Turn count: 5
Cumulative input tokens: 4790
Cumulative output tokens: 2429
Serialized conversation-history length: 13037 characters
=====
```

**The /stats command after turn 5 reports the updated cumulative token usage and history length.**

Final five-turn measurements are 5 turns, 4,790 cumulative input tokens, 2,429 cumulative output tokens and 13,037 characters of serialized conversation history. These measurements show how the conversation history and cumulative usage changed over the five turns.

## 4.5 Conceptual Questions

The important implementation idea is that the client keeps the conversation history and includes that history when making the next model request.

Theory: The client maintains the conversation by keeping the previous messages and sending them as context on later turns. This is what allows the model to respond with awareness of the earlier discussion.

Why is prior conversation context resent with every turn? The model needs previous messages to understand what has already been discussed, so the client sends the conversation history with each request.

How is a system prompt different from a user message? A system prompt establishes model behavior/instructions, while a user message contains the user's task or request.

Why do input tokens grow over a conversation? Previous user and assistant messages are included in later requests, increasing the amount of input context.

What eventually limits that growth? The model's context window limits how much conversation history can be included in one request.

## 5. Verification, AI Use, and Reproducibility

AI-use summary: I used AI assistance during development and review while I independently ran the commands, checked the screenshots and experiment outputs and compared the results with the assignment requirements. The first code-review output did not follow the required format so I corrected the client invocation and verified the revised review.

The repository contains the AI-use notes, run log, metrics, raw experiment data, fixed experiment input, README instructions and verification output needed to support the final submission.

```
(.venv) PS C:\Users\Bhushan Waingankar\Desktop\sjsu assignment
{
  "assignment": "DATA 260 Homework 1",
  "status": "PASS",
  "total_checks": 26,
  "passed_checks": 26,
  "failed_checks": 0,
  "checks": [
    {
```

**The final self-check reports PASS with 26 total checks, 26 passed, and 0 failed.**

## 6. Conclusion

Overall, this homework combined building, testing, and measuring an AI application. The web application established the domain context, the agent pipeline worked with that information, and the later experiments examined model behavior and the way the application communicates with the model. The testing and verification steps helped me check that the final implementation matched the required behavior.

Final repository information:

Final commit hash: 2921eec0d8badf6616d87af9f14a80d62970bc2

Git tag: hw1

GitHub repository: <https://github.com/shoddy16/data260-7085>